UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 9 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (12).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) A tightly coupled multiprocessor system shares a single main memory (a common global memory) that all processors access, and the processors communicate through that shared memory; a loosely coupled multiprocessor system gives each processor (or node) its own private local memory and the nodes communicate by passing messages over an interconnection network. In short, the major difference is the memory organisation/communication mechanism: shared common memory with tight coupling versus private per-node memory with message passing in loose coupling.

(b) The three types of cluster architecture are: (1) shared-disk cluster (all nodes share a common disk subsystem), (2) shared-memory cluster (nodes share a common memory), and (3) shared-nothing cluster (each node has its own private memory and disk, with no shared resources).

Unofficial marking guide:

• 1 mark: identifies that a tightly coupled system shares a single/common main memory (or communicates via shared memory)

• 1 mark: identifies that a loosely coupled system has private/local memory per processor/node and communicates by message passing (contrast clearly drawn)

• 1 mark: shared-disk cluster

• 1 mark: shared-memory cluster

• 1 mark: shared-nothing cluster

Model confidence: high.

Model note: Part (b) assumes the standard CSC429 classification of cluster architectures as shared-disk, shared-memory and shared-nothing; if the lecturer's notes use an alternative taxonomy (e.g. high-availability vs load-balancing vs high-performance), that wording should be substituted. No figure or table was required.

QUESTION A2 (5 marks)

Topic: Data representation and number systems

Answer: 100.510 = 0x42C90000 (sign=0, exponent=10000101, fraction=10010010000000000000000)

• 1 mark for the sign bit.

• 1 mark for the biased exponent.

• 1 mark for the 23-bit fraction.

• 1 mark for the correct 32-bit grouping.

• 1 mark for the final hexadecimal value.

QUESTION A3 (5 marks)

Topic: Computer generations and history

Source: CSC429 (13).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Inventor 'Y' is Blaise Pascal. The clue 'France's most celebrated mathematician and physicist', the year 1642, and the machine built to help his father with tax accounting all identify Pascal, who invented the Pascaline (arithmetic machine).

(b) Another calculating machine from the same generation (the mechanical calculating-machine generation, 17th century) is the Leibniz wheel / stepped reckoner (Leibniz calculator) invented by Gottfried Wilhelm Leibniz around 1673. (Accept the Pascaline's contemporary mechanical calculators such as the Leibniz calculator; do not accept later machines like the Difference Engine or Analytical Engine.)

(c) The programming language named after inventor 'Y' is Pascal.

(d) Pascal's calculating machine (the Pascaline) could perform addition and subtraction directly (with multiplication and division by repeated addition/subtraction). Two acceptable mathematical operations are: (1) addition, and (2) subtraction.

Unofficial marking guide:

© Hak Cipta Universiti Teknologi MARA

• 1 mark: (a) Correctly identifies inventor 'Y' as Blaise Pascal.
• 1 mark: (b) Names a valid same-generation calculating machine, e.g. the Leibniz calculator/stepped reckoner (or another contemporary 17th-century mechanical calculator).
• 1 mark: (c) States Pascal as the programming language named after inventor 'Y'.
• 2 marks: (d) One mark each for any two correct operations performed by the Pascaline, e.g. addition and subtraction (multiplication/division by repeated operations also acceptable).
Model confidence: high.
Model note: OCR errors in the question text (e.g. 'build', 'U^jj,', 'operations') were conservatively interpreted. The figure is not needed beyond the textual clues. For part (b), the 'same generation' is taken as the mechanical calculating-machine era; the Leibniz calculator is the standard accepted answer, though other contemporary mechanical calculators may be allowed.

QUESTION A4 (5 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (1).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The technique illustrated is direct addressing (also called absolute addressing). The instruction's address field A contains the actual/effective address of the operand, so the operand is fetched directly from memory location A — there is no intermediate pointer.

(b) How it is used: The instruction is fetched and its opcode decoded; the CPU takes the address field A as the effective address (EA = A) and places it on the address bus. The memory is read at location A and the operand returned on the data bus is used by the instruction (e.g. loaded into a register or used by an ALU operation). No additional memory reference is needed to compute the address, so one operand fetch follows the instruction fetch.

(c) Advantage: It is simple and fast — only one memory access is needed to obtain the operand (no extra indirection), and no address-calculation hardware is required. Disadvantage: The address field must be large enough to hold a full memory address, which lengthens the instruction; also the set of addresses is fixed at assembly time, giving limited flexibility (no easy relocation/pointer updating).

Unofficial marking guide:

• 1 mark: correctly names direct addressing (absolute addressing).
• 1 mark: states EA = A / address field holds the operand's address.
• 1 mark: explains the fetch sequence — instruction fetched, address A sent to memory, operand read and used.
• 1 mark: gives one valid advantage (e.g. single memory access / simple / fast).
• 1 mark: gives one valid disadvantage (e.g. long instruction / limited address range / inflexible fixed addresses).
Model confidence: high.
Model note: Figure 2 is only available as extracted text; the answer assumes the standard diagram in which the address field A points directly to the memory location holding the operand (no indirect pointer shown). If the figure actually shows a pointer/indirect box, the technique would be indirect addressing instead.

QUESTION A5 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (18).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) CPU involvement in interrupt-driven I/O versus DMA: In interrupt-driven I/O the CPU is directly involved in transferring the data. The CPU issues the I/O command to the device controller and then, while the device works, may continue with other instructions; when the device has a byte/word ready it raises an interrupt, and the CPU suspends its current program, runs the interrupt service routine, and itself moves the data between the device and memory (or a register). Thus the CPU must service one interrupt per unit of data transferred, so its involvement is high and grows with the amount of data. In Direct Memory Access (DMA) the CPU is involved only at the start and end: it programs the DMA controller with the memory address, the device address, the direction and the block count, then continues executing. The DMA controller performs the whole block transfer directly between the device and main memory, stealing memory cycles (cycle stealing) as needed, and interrupts the CPU only once when the entire block is complete. Hence CPU involvement in DMA is minimal and largely independent of block size, whereas in interrupt-driven I/O the CPU is heavily involved per data item.

(b) Definition of interrupt: An interrupt is a signal, generated either by an external device (hardware interrupt) or by the executing program/processor itself (software interrupt or exception/trap), that causes the CPU to temporarily suspend the program it is currently executing, save its state, and transfer control

to a designated interrupt service routine (handler); when the handler finishes, the CPU restores the saved state and resumes the interrupted program.

(c) What happens during a context switch: A context switch is the process of saving the complete state (context) of the currently running process and loading the saved state of another process so that the latter can run. The operating system saves the current process's context — program counter, CPU registers, status/flag register, and stack pointer — into its process control block (PCB), updates that process's state (e.g. to ready or blocked) and moves it to the appropriate queue. It then selects the next process to run, restores that process's context from its PCB into the CPU registers and program counter, and resumes execution of the new process. The switch is performed by the OS, typically triggered by an interrupt, trap or system call, and involves some overhead because useful work is not being done while the switch takes place.

(d) Benefit of a context switch: A context switch enables multitasking/multiprogramming — it lets the CPU be shared among several processes so that while one process waits for I/O or an event, another can use the CPU, improving CPU utilization and system throughput and giving the appearance of concurrent execution. It also supports responsiveness (interactive users get the CPU promptly), time-sharing/fairness, and the correct handling of interrupts and system calls, since the interrupted process can be resumed exactly where it left off.

Unofficial marking guide:

• 1 mark: states that in interrupt-driven I/O the CPU transfers/servicing data itself, one interrupt per data item, so CPU involvement is high

• 1 mark: explains that the CPU issues the command and continues until interrupted, then runs the ISR to move the data

• 1 mark: states that in DMA the CPU only sets up the DMA controller (address, count, direction) and is interrupted once at block completion, so involvement is minimal

• 1 mark: contrasts the two clearly (per-item CPU involvement vs block transfer with cycle stealing / single completion interrupt)

• 1 mark: defines interrupt as a signal (hardware or software) that suspends the current program

• 1 mark: states control transfers to an interrupt service routine/handler and the program resumes afterwards (state saved/restored)

• 1 mark: states that the current process's context (PC, registers, status, stack pointer) is saved into its PCB

• 1 mark: states that the next process's context is restored from its PCB and execution resumes (with OS selecting the process / updating states)

• 1 mark: benefit — enables multitasking/multiprogramming and better CPU utilization/throughput (or responsiveness/fairness)

• 1 mark: benefit — allows resuming the interrupted process correctly / supports interrupts and system calls, or explains overlap of I/O wait with useful CPU work

Model confidence: high.

Model note: The question writes 'Direct Manipulation Access'; this is taken to mean Direct Memory Access (DMA), the standard term. No figure or table is required.

QUESTION A6 (10 marks)

Topic: Data representation and number systems

a) Answer: 159910 = 63F16

• 1 mark for repeated division by 16.

• 1 mark for correct remainders.

• 1 mark for reading remainders in reverse order.

• 1 mark for the correct hexadecimal digits.

• 1 mark for the base subscript.

b) Answer: FD916 = 405710

• 1 mark for place values 16^2, 16^1 and 16^0.

• 1 mark for correct digit values.

• 1 mark for correct products.

• 1 mark for correct addition.

• 1 mark for the final decimal value.

QUESTION A7 (10 marks)

Topic: Input/output and interrupts

© Hak Cipta Universiti Teknologi MARA

Source: CSC429 (7).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Three types of interfaces to external devices [6 marks]

1. Character (byte-oriented) interface. This interface transfers data one byte (or character) at a time, typically over a serial or simple parallel link. It is used for slow devices such as keyboards, mice, printers and serial ports. Because each transfer is small, the I/O module must handle many individual transactions, often using interrupts to signal the CPU when a character is ready.

2. Block (block-oriented) interface. This interface transfers a block of data (many bytes/words) in a single operation, as used by disk drives, magnetic tapes and other mass-storage devices. It is efficient for large transfers and usually employs DMA so the I/O module moves the block directly to/from memory without per-byte CPU involvement.

3. Network (communication) interface. This interface connects the computer to a network (e.g. Ethernet, Wi-Fi) and handles packetised data with its own protocols and buffering. It is more complex than character or block interfaces because it must manage addressing, framing, error checking and asynchronous arrival of data, typically using interrupts and DMA.

(Other acceptable interfaces: memory-mapped vs. isolated (port-mapped) I/O, programmed I/O, interrupt-driven I/O, DMA-based interface, parallel vs. serial interface.)

(b) Classification of I/O devices to their I/O module [4 marks]

(i) Network interface card — network/communication I/O module (block/network interface).
(ii) Scanner — character (byte-oriented) input interface; slow input device.
(iii) Touch Screen — character (byte-oriented) input/output interface; slow interactive device.
(iv) Register — not an external I/O device; it is internal CPU/memory storage, so it does not belong to an external I/O module (it is accessed directly by the CPU, not through an I/O interface).

Unofficial marking guide:

- *1 mark: naming the first interface type (e.g. character/byte-oriented)*
- *1 mark: explaining the first interface type with a suitable example*
- *1 mark: naming the second interface type (e.g. block-oriented)*
- *1 mark: explaining the second interface type with a suitable example*
- *1 mark: naming the third interface type (e.g. network/communication)*
- *1 mark: explaining the third interface type with a suitable example*
- *1 mark: correctly classifying the network interface card to the network I/O module*
- *1 mark: correctly classifying the scanner to the character/byte-oriented input module*
- *1 mark: correctly classifying the touch screen to the character/byte-oriented I/O module*
- *1 mark: correctly identifying the register as internal storage, not an external I/O device*

Model confidence: medium.

Model note: The question text has OCR/formatting errors ('1/0' for 'I/O', stray 'iv)' and 'i)' labels). Assumed part (b) lists four items: (i) Network interface card, (ii) Scanner, (iii) Touch Screen, (iv) Register. Classification of scanner and touch screen as character-oriented interfaces is the conventional textbook answer; some schemes may accept 'input device'/'HID' grouping. The register item is a deliberate distractor: it is CPU/memory storage, not an external device.

QUESTION A8 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (11).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

The CPU registers each have a specific function in the fetch–decode–execute cycle. (1) Program Counter (PC): holds the address of the next instruction to be fetched; it is incremented (or updated by a branch) after each fetch so the CPU knows where to continue. (2) Instruction Register (IR): holds the instruction currently being decoded/executed after it is fetched from memory. (3) Memory Address Register (MAR): holds the address of the memory location to be read from or written to; it drives the address bus. (4) Memory Data Register (MDR) / Memory Buffer Register (MBR): holds the data word read from memory or the data word to be written to memory; it interfaces with the data bus. (5) Accumulator (AC): a general-purpose working register that holds one operand and receives the result of arithmetic/logic operations in a simple CPU. (6) General-Purpose Registers (GPRs): hold operands, intermediate results and addresses for the currently executing program, reducing memory accesses. (7) Status/Flag Register (Program Status Word, PSW): holds condition flags (zero, carry, sign, overflow) and control bits (interrupt enable, mode) that record the outcome of operations and CPU state. (8) Stack

Pointer (SP): holds the address of the top of the stack, used for subroutine calls/returns, interrupts and temporary storage. (9) Index Register: holds an offset used in indexed addressing to form an effective address. (10) Instruction Decoder (often considered part of control, not a data register): decodes the opcode in the IR to generate control signals. In a register-based answer, the essential set is PC, IR, MAR, MDR, AC/GPR, Status, SP and Index.

Unofficial marking guide:

• 1 mark: Program Counter (PC) — holds address of next instruction and is updated after fetch/branch
• 1 mark: Instruction Register (IR) — holds the instruction currently being decoded/executed
• 1 mark: Memory Address Register (MAR) — holds the memory address for a read/write and drives the address bus
• 1 mark: Memory Data Register (MDR/MBR) — holds data read from or to be written to memory and interfaces the data bus
• 1 mark: Accumulator (AC) — holds an operand and receives arithmetic/logic results
• 1 mark: General-Purpose Registers — hold operands, intermediate results and addresses
• 1 mark: Status/Flag Register (PSW) — holds condition flags and control bits recording operation outcomes and CPU state
• 1 mark: Stack Pointer (SP) — holds top-of-stack address for calls, returns, interrupts and temporary storage
• 1 mark: Index Register — holds an offset for indexed addressing to form an effective address
• 1 mark: Instruction Decoder/control register — decodes opcode in IR to generate control signals (or a further valid register such as a temporary register with correct function)
Model confidence: high.
Model note: The question text does not specify a particular CPU model; the answer assumes a simple von Neumann CPU with the standard register set. Some architectures combine MAR/MDR with cache or memory interfaces, and the instruction decoder is sometimes treated as part of the control unit rather than a register; the marking allows either a decoder or another valid register for the final mark.

QUESTION A9 (10 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (10).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Four storage access methods with examples: (1) Sequential access — data is accessed in a fixed linear order; example: magnetic tape. (2) Direct access — data is accessed by moving to a general location then sequentially searching; example: hard disk drive (or CD/DVD). (3) Random access — any location can be accessed directly in approximately the same time; example: main memory (RAM). (4) Associative access — data is accessed by content rather than address; example: cache memory (associative cache). [6 marks]

(b) With 11 address bits, the number of addressable memory locations = 2^11 = 2048 locations (2K locations). [2 marks]

(c) 2 MB = 2 × 2^20 bytes = 2^21 bytes. If the memory is byte-addressable, address bits = log2(2^21) = 21 bits. (If word-addressable with a different word size, the number of addressable units would change accordingly; assuming byte-addressable RAM.) [2 marks]

Unofficial marking guide:

• 1 mark: correct identification of sequential access method
• 0.5 mark: correct example for sequential access (e.g., magnetic tape)
• 1 mark: correct identification of direct access method
• 0.5 mark: correct example for direct access (e.g., hard disk)
• 1 mark: correct identification of random access method
• 0.5 mark: correct example for random access (e.g., RAM)
• 1 mark: correct identification of associative access method
• 0.5 mark: correct example for associative access (e.g., cache)
• 2 marks: correct calculation 2^11 = 2048 memory locations (1 mark for formula 2^n, 1 mark for correct result)
• 2 marks: correct conversion 2MB = 2^21 bytes and address bits = 21 (1 mark for conversion, 1 mark for log2 calculation)
Model confidence: high.
Model note: Assumption for part (c): RAM is byte-addressable. If word-addressable, the number of address bits would depend on word size. Part (a) examples are typical; other valid examples may be accepted.

QUESTION A10 (10 marks)

Topic: Computer architecture and CPU organisation

© Hak Cipta Universiti Teknologi MARA

Source: CSC429 (20).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The CPU is the processor that fetches, decodes and executes instructions; its main components and functions are:

• Control Unit (CU): Directs and coordinates all CPU and system activity. It fetches instructions, decodes the opcode, generates the timing and control signals that tell the ALU, registers and memory/bus which operation to perform, and sequences the machine cycle (fetch–decode–execute). It does not itself process data.

• Arithmetic Logic Unit (ALU): Performs all arithmetic operations (add, subtract, increment, etc.) and logical operations (AND, OR, NOT, XOR, shifts, comparisons). It takes operands from registers, produces a result, and sets condition/status flags (zero, carry, sign, overflow) in the status register.

• Registers: Small, fast storage inside the CPU. General-purpose registers hold operands and intermediate results; special registers include the Program Counter (PC), which holds the address of the next instruction; the Instruction Register (IR), which holds the instruction currently being executed; the Memory Address Register (MAR), which holds the address being accessed; the Memory Data/Buffer Register (MBR/MDR), which holds data read from or written to memory; and the Accumulator, which holds one ALU operand and the result. Registers provide the fastest level of the storage hierarchy.

• Internal buses / data paths: Move data and control signals between the CU, ALU and registers so that operands and results can be transferred during each step of the machine cycle.

• Clock and status/flags: The clock synchronises the steps of the machine cycle, and the flags register records the outcome of ALU operations for use by conditional branch instructions.

(b) The machine cycle is fetch → decode → execute (with memory read/write as needed). If the bus system is faulty, the address and data transfers between the CPU and memory/I/O cannot be completed reliably. In the fetch phase the PC value placed on the address bus may not reach memory, or the instruction read on the data bus may be corrupted, so the IR receives a wrong or invalid instruction and the CU decodes garbage. In the execute phase, operand fetches and result writes fail or return wrong data, so the ALU operates on incorrect operands and results are not stored. The CPU may then execute wrong instructions, hang, or raise a bus error/exception; the machine cycle cannot complete correctly and the program produces incorrect results or crashes.

Unofficial marking guide:

• 1 mark: Control Unit function (fetch/decode, control signals, sequencing)
• 1 mark: ALU function (arithmetic and logical operations, flags)
• 1 mark: General-purpose registers holding operands/results
• 1 mark: Special registers (PC, IR, MAR, MBR/MDR, accumulator) and their roles
• 1 mark: Internal buses/data paths moving data and control signals
• 1 mark: Clock/status flags role in synchronisation and conditional branching
• 1 mark: Overall CPU role of fetch–decode–execute and coordination of components
• 1 mark: Clear, accurate discussion with correct terminology across components
• 1 mark: Effect of faulty bus on fetch phase (address/instruction transfer corrupted)
• 1 mark: Effect on execute phase and overall machine cycle (wrong operands/results, bus error, program failure)

Model confidence: high.

Model note: Question text has a stray apostrophe in (a); treated as a full stop. The 8+2 split is assumed from the subpart marks given. No figure or table is required.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (17).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption on the equation (OCR correction): Figure 2 reads y = A(A + 2B/C-D 2). The only consistent reading with the available instructions (no exponentiation) is y = A × (A + 2B/C − D²), i.e. the trailing 2 is a superscript square on D. This is assumed throughout; if the figure shows a different expression, the operand order changes but the method is identical.

Sub-expression plan (common to all three programs): compute T1 = 2B/C, T2 = D×D, T3 = A + T1 − T2, then y = A × T3. A, B, C, D are read-only; x, y, z are temporaries; final result in y.

Zero-address program (stack machine):

```
PUSH A        ; stack: A
    PUSH B        ; stack: A, B
    PUSH B        ; stack: A, B, B
    ADD           ; stack: A, 2B       (B+B)
    PUSH C        ; stack: A, 2B, C
    DIV           ; stack: A, 2B/C
    ADD           ; stack: A + 2B/C
    PUSH D        ; stack: (A+2B/C), D
    PUSH D        ; stack: (A+2B/C), D, D
    MUL           ; stack: (A+2B/C), D*D
    SUB           ; stack: A + 2B/C - D*D
    PUSH A        ; stack: (A+2B/C-D*D), A
    MUL           ; stack: A*(A+2B/C-D*D)
    POP y         ; store final answer in y
```

One-address program (accumulator machine):

```
LOAD B        ; ACC = B
    ADD B         ; ACC = 2B
    DIV C         ; ACC = 2B/C
    ADD A         ; ACC = A + 2B/C
    STOR x        ; x = A + 2B/C   (save partial result)
    LOAD D        ; ACC = D
    MUL D         ; ACC = D*D
    STOR z        ; z = D*D
    LOAD x        ; ACC = A + 2B/C
    SUB z         ; ACC = A + 2B/C - D*D
    MUL A         ; ACC = A*(A + 2B/C - D*D)
    STOR y        ; y = final answer
```

Three-address program (register/operand form ADD(p,q,r) meaning p = q op r):

```
ADD (x, B, B)     ; x = 2B
    DIV (x, x, C)     ; x = 2B/C
    ADD (x, A, x)     ; x = A + 2B/C
    MUL (z, D, D)     ; z = D*D
    SUB (x, x, z)     ; x = A + 2B/C - D*D
    MUL (y, A, x)     ; y = A*(A + 2B/C - D*D)  final answer in y
```

Comment on correctness: In every version the read-only registers A, B, C, D are only ever read as source operands, the temporaries x and z hold intermediate values, and the final result is written to y, satisfying the stated constraints.

Unofficial marking guide:

• 4 marks: zero-address program — correct use of PUSH/POP with stack semantics, correct operand order for DIV and SUB, and final POP into y

• 4 marks: one-address program — correct LOAD/STOR/ADD/SUB/MUL/DIV sequence, correct use of temporaries x and z, final STOR y

© Hak Cipta Universiti Teknologi MARA

• 4 marks: three-address program — correct ADD/DIV/MUL/SUB with (p,q,r) destination-first convention, correct final MUL into y

*• 4 marks: correct evaluation of the expression (2B/C, D*D, A + 2B/C − D*D, then multiply by A) in all three programs*

• 2 marks: comments and correct assembly notation throughout, with A, B, C, D unmodified and final answer in y

• 2 marks: overall clarity, consistent register/temporary usage and no redundant or missing operations

Model confidence: medium.

Model note: The equation in Figure 2 is OCR-ambiguous ('D 2'); assumed to be D squared since no exponent instruction exists. The three-address table shows 'MUL (p.q.r)' — read as MUL(p,q,r). The (p,q,r) convention is assumed to be destination-first (p = q op r); if the lecturer uses source-first ordering, swap the operand positions accordingly. Figure 2 itself was not available beyond the extracted text.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA